

LEARN BY READING A REAL PROJECT

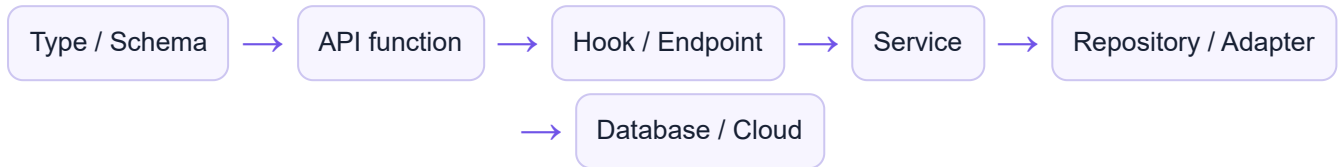
IntelliQR Coding Learning Guide

Every feature explained from its smallest code modules to its complete frontend–backend workflow.

For learning React, TypeScript, FastAPI, MongoDB, security, QR processing, storage and deployment

How this guide teaches the project

This document does not only list features. It teaches how each feature is assembled from small reusable modules. Read in order:



Learning rule: Start with one small function and follow its input and output. Do not begin by trying to understand an entire large page such as `QrGeneratorPage.tsx`.

Five questions to ask when reading any module

1. What input does it receive?
2. What output does it return?
3. Which rule belongs here?
4. Which lower-level module does it call?
5. Who calls this module?

Recommended learning path

- 1 Fundamentals
- 2 Authentication
- 3 Basic QR
- 4 Scanner
- 5 Dynamic tracking
- 6 Encryption
- 7 Cloud storage
- 8 Batch/admin/deployment

1. The smallest frontend building blocks

Module	What it teaches	Used by
<code>frontend/src/lib/cn.ts</code>	Small utility functions and class composition.	Reusable UI and pages.
<code>components/ui/Button.tsx</code>	Typed props, variants, loading and icons.	Every page action.
<code>Card.tsx</code> , <code>Badge.tsx</code> , <code>Input.tsx</code>	Composition, consistent design system and accessibility.	Forms, dashboards and public access pages.
<code>Spinner.tsx</code>	Reusable loading feedback.	Lazy routes and asynchronous data.
<code>features/*/types</code>	TypeScript interfaces as contracts.	API modules, hooks and pages.

Example mental model: Button

A page should not repeatedly code borders, disabled states, spinners and icon positions. A reusable Button accepts properties such as `variant` , `size` , `loading` and `icon` . This is the React principle of **composition**: pages describe intent while the component owns presentation.

Practice: Open `Button.tsx` . Find its prop interface, default values and conditional class names. Then search `<Button` to see how many larger features reuse it.

Why types come before pages

For example, `QrGeneration` describes the data returned by the backend. A hook can promise `Promise<QrGeneration>` , and the page receives autocomplete and compile-time errors if it uses a missing property.

2. The smallest backend building blocks

Folder	Coding concept	Example
core/config.py	Configuration object and environment validation.	Settings
core/exceptions.py	Domain-specific failures separate from HTTP code.	NotFoundError
models/	Domain state represented as dataclasses.	QrGeneration
schemas/	Boundary validation and serialization.	UrlQrRequest
repositories/	Database access isolated from business logic.	UserRepository
services/	Use cases and business rules.	AuthService
infrastructure/	External technical details.	OpenCV, Blob, Brevo
api/v1/endpoints/	HTTP request/response translation.	auth.py

Schema versus model

A schema describes data entering or leaving the API and can validate it. A model describes application/domain state after loading from MongoDB. Keeping them separate prevents HTTP-specific choices from spreading through business logic.

Exception flow

```
Service detects missing resource
  → raises NotFoundError
  → main.py exception handler
  → returns safe JSON + HTTP 404 + request ID
```

Practice: Find one `raise NotFoundError` in a service, then find its handler in `main.py`. This teaches exception propagation.

3. How frontend and backend communicate

Shared frontend client

`frontend/src/lib/api/client.ts` wraps `fetch`. It prevents every feature from repeating URL construction, JSON headers, cookies, Bearer tokens, error parsing and refresh retry.

```
Page → feature hook → feature API → apiClient → fetch('/api/v1/...')
```

Small example

```
// Feature API describes the request
get(id): Promise<QrGeneration> {
  return apiClient.get(`/qr/generations/${id}`)
}

# Endpoint validates authentication and parameters
@router.get('/{generation_id}')
async def get_generation(...):
  return await service.get_owned(generation_id, current_user.id)
```

Vercel production rewrite

The browser calls the same Vercel origin. `vercel.json` forwards `/api/:path*` to Render. This teaches the difference between an application route, an API route and a reverse proxy/rewrite.

Important: The browser should not receive MongoDB, Blob or Brevo secrets. It communicates only with FastAPI, and FastAPI communicates with those services.

4. Dependency injection: how modules are connected

`backend/app/api/dependencies.py` is the composition root. It answers: “Which concrete objects does this feature need?”

```
def get_qr_generator_service(repository, settings):
    return QrGeneratorService(
        repository=repository,
        renderer=QrRenderer(),
        storage=VercelBlobQrStorage(...) or LocalQrStorage(...),
        payload_builder=QrPayloadBuilder(),
        ...
    )
```

The generator service does not decide how FastAPI creates its repository. It only receives an object with the required operations. This is **dependency inversion**.

What this design enables

- Production can use Vercel Blob while local development uses disk.
- Tests can inject a fake repository or fake Blob client.
- Business logic is not tied to FastAPI global variables.
- Features can be changed at the application boundary.

Practice: Trace `get_share_vault_service`. Write down every dependency it creates and whether it belongs to data, security, storage or analytics.

5. Authentication feature from small to large

Step	Module	Responsibility
1	features/auth/types/auth.ts	Login/register/user TypeScript types.
2	features/auth/api/authApi.ts	Calls register, login, refresh, logout and current-user endpoints.
3	features/auth/context/AuthContext.tsx	Stores current user and exposes actions to the app.
4	LoginPage.tsx , RegisterPage.tsx	Forms, validation feedback and navigation.
5	schemas/auth.py	Validates API request bodies.
6	endpoints/auth.py	Maps HTTP calls to AuthService.
7	services/auth_service.py	Normalization, password checks, account state and token workflow.
8	core/security.py	Bcrypt and JWT operations.
9	user_repository.py , session_repository.py	MongoDB account/session persistence.

Learning React Context

`AuthProvider` owns `user` and `isInitializing`. It exposes actions through a context so `Navbar`, `ProtectedRoute` and pages share one authentication state without passing props through every component.

Learning refresh restoration

On mount, `useEffect` calls `restoreSession()`. The `HttpOnly` cookie travels automatically. The backend rotates it and the provider sets the returned user. A cleanup flag prevents updating state after unmount.

Practice: Add a temporary console statement in each `AuthContext` action locally. Log in, refresh and logout to observe the lifecycle, then remove the logs.

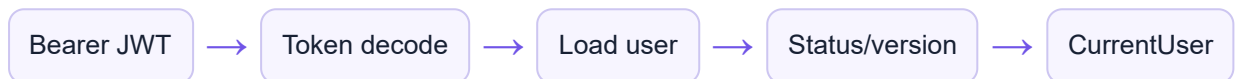
6. Protected routing and role checks

Frontend layer

`App.tsx` declares public and protected routes. `ProtectedRoute` reads `AuthContext`. It first waits for initialization, then redirects anonymous users, and can require an admin role.

Backend layer

`get_current_user` decodes the Bearer access token, loads the user, checks active status and compares `token_version`. `get_current_admin` adds the role condition.



Security lesson: Frontend protection improves UX but is not authority. Backend dependencies must enforce access because HTTP endpoints can be called without the UI.

7. Build a basic QR generator first

Smallest possible idea

```
text → QR matrix → draw modules → PNG bytes
```

In IntelliQR, `QrPayloadBuilder` converts structured form data to a standard string. `QrRenderer` uses `qrcode` to create the matrix and `Pillow` to draw the output.

Modules to read in order

1. `schemas/qr_generation.py` — supported requests and rules.
2. `services/qr/payload_builder.py` — URL/text/mailto/tel/Wi-Fi/vCard/geo strings.
3. `infrastructure/qr/renderer.py` — matrix and image drawing.
4. `services/qr/generator_service.py` — complete use case.
5. `endpoints/qr_generations.py` — REST interface.
6. `features/qr-generator/api` and `hook` — frontend connection.
7. `QrGeneratorPage.tsx` — full UI composition.

Why PayloadBuilder is separate

Creating correct Wi-Fi or vCard text is different from drawing black squares. Separation lets payload syntax be tested without rendering an image.

Practice: Call `QrPayloadBuilder().build(TextQrRequest(...))` in a Python shell. Then try Email and Wi-Fi types and observe the standardized strings.

8. BrandCraft: learning image rendering

BrandCraft adds design after the basic QR works. The request schema validates color format, contrast, size, margin, frame text and error correction.

Renderer subproblems

Function idea	What it teaches
<code>_matrix</code>	Use a library for QR encoding while retaining control of drawing.
<code>_layout</code>	Coordinate systems, module size and centering.
<code>_module_color</code>	RGB interpolation for gradients.
<code>_is_finder</code>	Preserve finder patterns for reliability.
<code>_paste_logo</code>	Image normalization, scaling and alpha composition.
<code>_render_svg</code>	Generate vector markup manually.
<code>_render_pdf</code>	Convert raster output for document export.

Frontend preview versus backend output

`BrandedQrPreview.tsx` gives immediate visual feedback in the browser. The backend is still authoritative for saved/downloadable files. Both use the same design contract, demonstrating how shared data rules align two renderers.

9. Persisting and managing QR records

After rendering, `QrGeneratorService.generate` stores artifacts and creates a MongoDB record through `QrGenerationRepository`. The record includes ownership, safe payload preview/details, slug, design, file paths, limits and timestamps.

Why store a preview and hash?

- Preview lets the Dashboard display safe information.
- Hash supports identity/deduplication ideas without exposing full sensitive content.
- Protected destinations use encrypted storage instead of plaintext.

Dashboard module chain

```
DashboardPage
→ useQrGenerations / useDashboardSummary
→ qrGeneratorApi / dashboardApi
→ endpoints
→ QrGeneratorService / DashboardService
→ QR and Dashboard repositories
→ MongoDB aggregations
```

Editing a design re-renders and stores replacement files before deleting old storage. This teaches safe replacement ordering: create the new asset first, update the database, then remove the previous asset.

10. Durable QR downloads and storage adapters

Local adapter

`LocalQrStorage` writes to a configured directory. It is easy for development but unsuitable for free Render persistence.

Cloud adapter

`VercelBlobQrStorage` saves private objects, reads authenticated bytes and deletes the generated group. Both adapters expose conceptually similar `save`, `read` and `delete` operations.

Download flow

Download button

- `qrGeneratorApi.saveDownload()`
- authenticated GET file endpoint
- service loads record and reads private Blob
- FastAPI streams bytes with Content-Disposition
- browser creates object URL and saves file

Abstraction lesson: The generator service asks storage to save/read/delete. It does not contain Vercel HTTP details. This keeps business logic independent of the vendor.

11. Build a basic scanner, then extend it

Basic scanner

```
image bytes → cv2.imdecode → QRCodeDetector → decoded string
```

Project modules

Module	Addition
infrastructure/qr/decoder.py	Image validation and QR decoding.
services/qr/classifier.py	Recognizes the content type.
services/qr/safe_scan.py	URL risk heuristics.
services/qr/scanner_service.py	Stores owner history.
endpoints/qr_scans.py	Upload/analyze/list/delete HTTP operations.
useQrScanner.ts	Frontend scanning state and errors.
QrScannerPage.tsx	Drag/drop, file input, camera and results.

Camera lesson

The browser obtains a media stream, draws a video frame to a canvas and converts it to a Blob. The backend receives it exactly like a normal uploaded image. The camera does not decode the QR itself in this architecture.

Practice: Temporarily test only `decoder.py` with one local QR image. After that works, trace the multipart endpoint and React FormData upload.

12. Dynamic QR tracking

Start with a redirect

A dynamic QR encodes `{redirect_base_url}/r/{slug}`. `qr_redirects.py` receives a public GET, asks `QrGeneratorService.resolve_redirect` for the destination and returns HTTP 307.

Add tracking carefully

1. Find the generation by unique slug.
2. Check active, expiry and scan limit.
3. Atomically increment scan count.
4. Create an analytics event as a background task.
5. Redirect without caching or referrer leakage.

Why configuration matters

The hostname is permanently encoded in the QR image. `INTELLIQR_REDIRECT_BASE_URL` must be reachable by phones and spelled exactly. Fixing the environment changes only newly generated images.

Debugging lesson: Never guess what a QR contains. Decode the actual downloaded image, print the embedded URL and test that URL independently.

13. Analytics from events to charts

Event creation

`AnalyticsService.record_scan` receives the generation, client IP and user agent. `ScanContextService` converts these into privacy-safe dimensions and a keyed visitor hash.

Aggregation

`AnalyticsRepository` uses MongoDB pipelines to group events by date, QR, device, browser, OS and location. The service fills missing date points and calculates comparison percentages.

Frontend chart chain

```
AnalyticsPage
→ useAnalytics(period, qrId)
→ analyticsApi
→ analytics endpoint/service/repository
→ ScanTrendChart + BreakdownList
```

Coding lessons

- Event sourcing for reporting.
- Aggregation rather than loading every record into Python.
- Derived values belong in service/repository output, not hard-coded UI.
- Charts should display real empty states instead of fake sample data.

14. AES-GCM encryption module

The central module is `backend/app/core/vault.py`. `VaultCipher` derives a 256-bit key with SHA-256, generates a new 12-byte nonce and uses `AESGCM`.

```
nonce = secrets.token_bytes(12)
encrypted = cipher.encrypt(nonce, plaintext, associated_data)
stored = nonce + encrypted
```

Why associated data differs

- `intelliqr-destination-v1` binds encrypted strings to the destination format.
- `intelliqr-file-v1` binds encrypted bytes to file storage.

Associated data is authenticated but not encrypted. Using distinct labels prevents one ciphertext type from being treated as another.

Module users

User	Purpose
<code>QrGeneratorService</code>	Encrypt protected destination URLs.
<code>VercelBlobShareStorage</code>	Encrypt/decrypt file bytes around Blob operations.
<code>EncryptedShareStorage</code>	Same behavior for local files.
<code>dependencies.py</code>	Constructs the cipher with the Vault key or fallback secret.

Cryptography lesson: Never reuse a nonce with the same AES-GCM key. The code generates a fresh random nonce for every operation. Losing the encryption key makes old content unreadable.

15. SecureVault: policy plus short-lived grants

SecureVault is learned best as two stages: **authorize**, then **redirect**.

Stage 1: public access page

`VaultAccessPage.tsx` loads policy information, shows password/sign-in UI and calls `vaultApi`.

`VaultAccessService` verifies the password, authenticated user or allowlist.

Stage 2: grant

`VaultGrantService` creates a short-lived JWT containing subject, slug, purpose, type, ID, issued/expiry time, issuer and audience. The redirect validates every claim before decrypting the destination.



Authorization lesson: A grant is proof that a recent policy check succeeded. It is short-lived and bound to one resource rather than being a permanent download/redirect secret.

16. ShareVault from upload to download

Owner-side modules

`ShareVaultPage.tsx` collects a file and policy. `useShares.ts` controls state. `shareVaultApi.ts` sends multipart FormData. `shares.py` accepts the upload and calls `ShareVaultService.create`.

Creation inside the service

1. Validate extension, size and signature.
2. Validate password/allowlist/expiry/limit.
3. Create random slug and storage key.
4. Encrypt and upload through the storage adapter.
5. Create a direct QR to the Vercel access page.
6. Create the MongoDB metadata record.
7. If a later step fails, delete already-uploaded bytes (rollback).

Recipient-side modules

`SharedFileAccessPage.tsx` loads public policy, obtains a grant and downloads. The backend rechecks policy identity, reads encrypted Blob bytes, decrypts in memory, atomically consumes a download and records history.

Practice: Trace one public share and one private share. Write which requests require authentication and why.

17. File validation and secure responses

Validation examples

- PDF must begin with `%PDF` .
- Office/ODT containers must begin with ZIP marker `PK` .
- Images are opened and verified with Pillow.
- MP4/MOV checks `ftyp` ; WebM checks its signature.
- Text/CSV must decode as UTF-8.

Response headers

Header	Purpose
<code>Content-Disposition: attachment</code>	Downloads with a safe original/generated filename.
<code>Cache-Control: no-store</code>	Avoids caching decrypted/private bytes.
<code>Referrer-Policy: no-referrer</code>	Prevents grant URL leakage.
<code>X-Content-Type-Options: nosniff</code>	Stops browsers guessing an unsafe content type.

Boundary lesson: Extension/signature checks reduce obvious abuse but are not antivirus scanning. Code and UI should never claim stronger protection than implemented.

18. BulkForge: reuse, jobs and partial success

Modules in order

1. `BulkForgePage.tsx` — upload, format selection and job table.
2. `useBulkJobs.ts` — polling and actions.
3. `bulkForgeApi.ts` — multipart/API calls.
4. `infrastructure/bulk/parser.py` — CSV/XLSX rows to QR request schemas.
5. `BulkForgeService` — progress and row loop.
6. `QrGeneratorService` — reused individual generation.
7. `BulkZipStorage` — safe ZIP names and bytes.
8. `BulkJobRepository` — persistent job state.

Why background tasks?

The create endpoint responds with HTTP 202 and schedules processing. The frontend polls status instead of holding one long upload request. For very large production workloads, a real queue/worker would replace in-process background tasks.

Partial result pattern

Each row is isolated in a try/except. Valid rows continue, failed rows add bounded messages, and the final status can be `partial`. This is a useful design for imports.

19. Dashboard aggregation

A dashboard is not a single table. `DashboardRepository` queries multiple collections and returns a summary. `DashboardService` presents it through the endpoint. The frontend hook loads it and the page maps values to stat cards and lists.

Learning aggregation

- Count close to the database instead of loading all objects.
- Restrict every aggregation by owner ID.
- Keep recent lists bounded.
- Record lightweight download events rather than duplicating full files.
- Render honest empty states.

Practice: Choose one Dashboard card. Trace its number back to the exact repository query and collection.

20. Notifications and Brevo email

Event creation pattern

Feature code calls `NotificationService.notify(user_id, event_type, category, severity, title, message, ...)`. The service checks category preferences, creates an in-app record and optionally sends email.

Delivery adapter

`BrevoEmailSender` uses `httpx.AsyncClient` to POST sender, recipient, subject, text and branded HTML. It escapes title/body before HTML insertion. If no Brevo key exists, dependency injection can select local SMTP for development.

Why notification failure does not fail the main action

Email is a secondary side effect. A successful ShareVault download should not be reversed because the email provider is temporarily unavailable. Failures are caught and logged.

Reliability lesson: Separate the primary transaction from optional notifications. In a larger system, an event queue would improve retry and delivery guarantees.

21. Profile and password rotation

`ProfilePage.tsx` contains separate display-name and password workflows. `ProfileService` verifies business rules and calls User/Session repositories.

Password rotation sequence

1. Verify the current password hash.
2. Validate and hash the new password.
3. Update the user and increment token version.
4. Revoke all refresh sessions.
5. Clear the refresh cookie.
6. Frontend returns to login.

This feature teaches that changing a password is not only updating one database field; existing credentials must be invalidated.

22. Admin and audit features

Module chain

```
AdminPage → useAdminDashboard → adminApi
→ admin endpoints (AdminUser dependency)
→ AdminService
→ AdminRepository + SessionRepository
→ users and admin_audit_events collections
```

Business rules in the service

- An admin cannot disable/change their own access accidentally.
- The final active administrator cannot be removed.
- Role/status changes revoke target sessions and invalidate tokens.
- Every sensitive change creates an audit event.

Layering lesson: These rules belong in the service, not only in disabled frontend buttons, because API callers can bypass the page.

23. Internationalization and theme

i18n modules

`translations.ts` maps stable keys to English, Hindi and Gujarati strings. `LocaleContext` owns selected locale, and `useLocale` provides translation access. Account locale is updated through a user endpoint.

Theme modules

`useTheme.ts` manages theme state and applies a DOM attribute. `index.css` defines theme-specific visual rules. Components remain reusable because they use semantic classes/tokens instead of page-specific inline colors wherever possible.

Practice: Add one new translation key to all three dictionaries and render it on a test component. This shows why stable keys must exist in every locale.

24. Database repositories and indexes

Repository pattern

A repository translates domain operations into MongoDB queries. Example: `find_owned(id, user_id)` includes both identifiers, so ownership is part of the query rather than checked later.

Why indexes are created at startup

`db/mongodb.py` defines unique and compound indexes for emails, slugs, owner/time lists, analytics and unread notifications. Indexes turn common filters/sorts from collection scans into efficient lookups.

Repository learning sequence

1. Read a simple `find_by_id`.
2. Read `create` and document-to-model conversion.
3. Read owner-filtered pagination.
4. Read atomic update/consume logic.
5. Finally read analytics aggregation pipelines.

25. Error handling and observability

Request IDs

Middleware accepts or generates `X-Request-ID` , adds it to response headers and structured logs. A frontend error can be matched with a backend log without exposing stack traces.

Error categories

Type	Response
Application rule	Specific safe code/message and appropriate status.
Pydantic validation	422 with validation details.
Authentication	401 and Bearer challenge.
Unexpected exception	Generic 500; full exception only in server log.

Structured logging

Logs include timestamp, level, logger, message and contextual fields such as path, status and duration. This is easier for hosted log search than arbitrary print statements.

26. Testing each layer

Test type	Project examples	Learning goal
Pure unit	PayloadBuilder, classification, validation	Test one rule with no external state.
Adapter fake	Vercel Blob and Brevo fake clients	Verify requests and round trips without network.
Service test	Share grants and authentication	Test workflows with fake repositories.
API contract	FastAPI TestClient tests	Validate status and response shape.
Frontend API	Vitest feature API tests	Verify methods, paths and bodies.
Build/type check	TypeScript + Vite	Catch contract and bundling failures.

Arrange–Act–Assert

```
# Arrange fake repository/settings
# Act by calling the service
# Assert result, stored values and side effects
```

Practice: Read `test_vercel_blob_gr_storage_round_trip` . Identify the fake client, action and assertions. Then create a failing case.

27. Local development and deployment coding lessons

Local commands

```
# Backend
cd backend
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000

# Frontend
cd frontend
npm run dev -- --host 0.0.0.0
```

What `.venv` teaches

A virtual environment isolates Python packages. `requirements.txt` declares dependencies, and Render rebuilds them in a clean environment instead of copying the developer's virtual environment.

Environment-aware adapters

The same source runs locally and in production because Settings supplies URLs/secrets and dependency injection chooses local or cloud storage. This teaches the twelve-factor principle of external configuration.

Deployment chain

```
git commit → GitHub main
→ Render backend build/deploy
→ Vercel frontend build/deploy
→ health + end-to-end smoke tests
```

28. Common bugs and what they teach

Bug	Root cause	Learning
Refresh returns to auth	Route decision before session restoration.	Async initialization needs an explicit loading state.
Backend Online mismatch	Wrong host, CORS or cold start.	Test actual health route from the browser environment.
QR opens Not Found	Misspelled hostname encoded permanently.	Inspect generated artifacts, not only configuration.
Old download missing	Ephemeral Render disk.	Metadata persistence and byte persistence are separate.
Blob empty	Wrong prefixed environment variable.	Configuration names are part of the application contract.
SMTP fails	Hosting blocks SMTP ports.	Infrastructure constraints affect architecture; HTTPS API is an alternative.
Share QR backend page	Unnecessary redirect layer.	Use the simplest URL flow that meets tracking requirements.

29. Feature-to-module master map

Feature	Frontend	Backend service/endpoint	Data/adaptor
Authentication	AuthContext, authApi, Login/Register	AuthService, auth endpoints	User/Session repositories, JWT/bcrypt
QR generation	QrGeneratorPage, hook/API, preview	QrGeneratorService, generation endpoints	PayloadBuilder, Renderer, QR repository, Blob
QR management	DashboardPage, useQrGenerations	QrGeneratorService	QR/Download repositories
Scanner	QrScannerPage, useQrScanner	QrScannerService, scan endpoints	Decoder, Classifier, SafeScan, scan repository
Tracking	Analytics page	redirect endpoint, AnalyticsService	Analytics repository, ScanContext
SecureVault	Generator controls, VaultAccessPage	VaultAccessService, redirect	VaultCipher, grant JWT, QR repository
ShareVault	ShareVaultPage, SharedFileAccessPage	ShareVaultService, share endpoints	Share repositories, AES-GCM, private Blob
BulkForge	BulkForgePage, job hook/API	BulkForgeService, bulk endpoints	Parser, QR generator, ZIP, job repository
Notifications	NotificationsPage, hook/API	NotificationService	Notification repository, Brevo sender
Admin	AdminPage, admin hook/API	AdminService, admin endpoints	Admin/User/Session repositories
Profile	ProfilePage	ProfileService, user endpoints	User/Session repositories, bcrypt
i18n/theme	LocaleContext, useTheme, CSS	Locale profile endpoint	User locale field/browser preference

30. Build IntelliQR again as learning exercises

Level 1 — beginner

1. Create a React file input and preview filename.
2. Create a FastAPI health endpoint.
3. Upload an image with FormData.
4. Decode one QR with OpenCV.
5. Create one text QR and return PNG.

Level 2 — intermediate

1. Add MongoDB persistence and owner IDs.
2. Add login with bcrypt and JWT.
3. Create reusable API client and hooks.
4. Add QR history, delete and downloads.
5. Add direct versus dynamic URL mode.

Level 3 — advanced

1. Add scan events and aggregation analytics.
2. Add AES-GCM encrypted destinations and short grants.
3. Add encrypted private file storage.
4. Add batch jobs with partial results.
5. Add role-based admin and audit history.
6. Add external email through an adapter.

Level 4 — production

1. Use durable object storage.
2. Move secrets to hosting environment variables.
3. Configure exact origins, cookies and rewrites.
4. Add unit, contract, storage and smoke tests.
5. Decode deployed QR artifacts and test from another device.

31. Final coding principles learned

Separation of concerns

Pages display, hooks orchestrate, APIs transport, endpoints adapt HTTP, services decide, repositories persist and infrastructure integrates.

Single responsibility

PayloadBuilder creates payload syntax; Renderer draws; Storage stores. Small focused modules are easier to test.

Dependency inversion

Services depend on behavior, while dependency injection chooses local/cloud implementations.

Defense in depth

Frontend guards, backend authorization, validation, encryption, safe headers and private storage reinforce one another.

Data ownership

Owner filters belong in database queries, not only after retrieval.

Honest limitations

SafeScan is heuristic, free hosting sleeps and unauthenticated email domains have deliverability limits.

Test behavior

Fake external clients but assert the request, data transformation and side effects.

Debug evidence

Inspect actual QR contents, response codes, logs, environment names and deployed commit IDs.

Conclusion

IntelliQR is best understood as a collection of small, testable modules connected into larger workflows. Begin with types and pure functions, follow one request through every layer, and only then read the large page or service as a whole. That method can be reused for almost any full-stack project.

End of IntelliQR Coding Learning Guide · Generated from the current project modules.